

What am I looking for in an application?

- My ideal application is one that **teaches me something new**.
 - This looks like identifying an interpretability hypothesis, gathering evidence for and against it, and writing up the evidence and analysis clearly.
- I value clear writing, good taste (ie choosing interesting problems and making good decisions), technical skill, truth-seeking, skepticism and pragmatism
- See a much more detailed explanation in [this tab](#), along with past examples
- A significant factor is whether I find your question and research direction interesting - in addition to the technical quality of the work. I'm afraid my interests are somewhat vague and "I know it when I see it" (sorry!), so please read [how my research interests have changed](#) before choosing a problem.

Can I use LLMs?

- Yes. In fact, I strongly recommend it! **LLMs are a crucial research tool** nowadays, and are especially useful for those getting into a new field.
 - More advice on using LLMs well [below](#)
- I want this to be a faithful test of how you'd use them if doing research for me, so I want you to use them however you would
- It is your responsibility to ensure your code and writing are high quality. Well-written write-ups are welcome. Docs that read like LLM slop will be rejected.
- In particular, using an agentic coding tool is highly recommended: my top recommendation is Claude Code with Fable (worth paying for the Max plan for the application period if you can afford it). GPT 5.6 Sol in Codex, or Opus 5 in Claude Code, are also solid options. You also want a great chatbot to help explain concepts, give interpretability context, etc.
- Crucially: sanity-check everything your agent does. A key thing I'm evaluating is whether you add value beyond me just prompting Fable myself - see Sanity-check your agent for what this looks like. Docs that read like LLM slop will be rejected.
- I've compiled a [folder of useful text files](#) for mech interp research, containing a bunch of relevant docs & source code of key libraries, tutorials from ARENA and key libraries, key papers and my relevant blog posts.
 - By default, just **put [this 600k token file](#) in the context window**, which contains the most important documents¹.

Suggested Research Problems

The below are a bunch of recommendations for things I would be excited about. Strong applications often riff off of these ideas - coming up with their own approach, but along similar themes to the below. You should not feel constrained to the problems on this list, but hopefully it can serve as some guidance for the types of questions I'd be excited to see.

¹ It starts with a table of contents explaining what's in it.

Warning: The ideas below have **not** been filtered for “I am confident someone could make progress on this in 20 hours”. Pick something where you have some idea of how to get started (or read around the field a bit and try to generate ideas and a sketch plan before picking a problem), and expect to need to scope the ambition down as the project goes on. Do not trust LLM time estimates, in my experience they’re super off

Model Forensics

A particularly important application of model biology is [model forensics](#): when a model has taken a seemingly sketchy action, can we figure out the motivations, especially whether it was true misalignment or has a benign explanation like confusion. Please look at [our paper](#) if you want to do a project here, there’s a lot of relevant advice!

- Good settings for sketchy behavior are included in our [model forensics paper](#) and [task gaming blog post](#)
 - Rich datasets that may contain interesting things are [weird chat](#) (weird behaviors automatically found in open models) and [SWE-chat](#) (real agent transcripts of various models)
 - It’s OK but not ideal to study a transcript on a model other than the one that made it. Only do this if resampling with the new model recreates the weird behavior
- As discussed in our paper, the strongest techniques here seem to be reading the chain of thought to form hypotheses, and constructing precise changes to causally test counterfactuals
 - I’m pretty interested in whether you can take a setting where the model acts plausibly deceptively and this does *not* work, and see if you can understand this better with more involved techniques (chain of thought resampling, internals based methods, etc)
- Good projects here look like taking some instance of sketchy behaviour and doing a deep dive into what’s going on and what drove it, and trying to form rigorously backed conclusion
 - My alum Aditya Singh [suggests several good projects here](#)
- Can you make a good agent to automatically do forensics for you? See [auditing agents](#) and [petri](#) for an open source starting point
- The idea of using [synthetic document finetuning](#) to teach the model false facts, and [using these to perform better forensics](#), is fascinating to me - how useful is this? How well does it work? What else can you do with it?
- One major issue in forensics is eval awareness. Can you better characterize [eval awareness in the best open source models](#)?
 - How well does [steering against it](#) work for mitigating it? What are the side effects? Should we trust this in a real eval?
 - How well can we probe for it in real models? If we ran this over evals where only 1% showed eval awareness, would we just get tons of false positives?

- I'm also interested in deep dives into examples of weird but not necessarily misaligned behavior and trying to really understand what's going on, and ideally fix it? Example: debugging [why it thinks 9.8 > 9.11](#)

Application Format

Format: An application should consist of a summary of your findings in the [application form](#), and a google doc describing your key findings which begins with an executive summary and ideally contains a bunch of graphs, and enough detail to follow what you did without needing to read your code. You're encouraged to include code, but it's not required, I'll largely use it to give my agents context and ask them questions about what you actually did and I'll only read it as necessary to understand the write-up better. **Remember to let anyone with the link access the doc!**

Write-ups are important so you get 2 extra hours to do it. Prioritize the application form summary Qs, I read these first and use them as a preliminary filter, I don't have time to read every write-up. Convey concretely what you did, what you found, why it's interesting, biggest limitations, etc. Specifics beat vibes: name the models, the key experiment, the surprising number.

Please do not just submit raw LLM output for the application form or executive summary. Write these yourself, in your own voice, even if you think an LLM will sound better. Trust me: even if it seems like it'll be better to have an LLM do it, using an LLM will make it sound eerily similar to all the other LLM-written applications, in ways you won't notice. Answers that read like they were written by an LLM are a significant negative signal - I see hundreds of them, and they blur together.

Executive Summary Format

The first 1-3 pages of the google doc should be an executive summary, which gives the broad strokes of what you did and what you learned. Something at **~1 page** (including graphs) is great, **max 3 pages** and **max 600 words**. Please **include graphs!** Bullet points can work well

One good format is to have sections for:

- What problem am I trying to solve? (and a bit on why you think it's interesting)
 - Remember - what you write is always far clearer to yourself than to the reader! Though you can assume it will be read by someone with mech interp research experience
- What are your high-level takeaways? What were the most interesting parts of your project?
- One paragraph and graph per key experiment, giving the gist of what it was, what you found, and why this supports your key takeaways

If bad data would sink your project, show me the data. If everything rests on the quality of some dataset or judgement calls (e.g. you generated the dataset with an LLM, or used an LLM judge

to score outputs), look at it yourself - and include some randomly selected qualitative examples in the write-up, ideally just after the executive summary. Randomly selected, not cherry-picked! A handful of raw examples is the easiest way to show me that the thing your whole project rests on is actually real.

Can I submit relevant research I've already done?

- If you've previously done relevant research (mech interp, or other AI safety work along the lines of my research interests) i.e. a (co-)first author² paper or non-first author but significant contribution paper or high-effort blog post, I'm open to you writing an executive summary for that work, and linking to it, rather than doing the normal application.
 - Please include an **estimate of how many hours the project took you**
 - If other people worked on it with you, please include a description of what you specifically contributed
 - If it's not obviously a mech interp paper then please explain why you think it shows you have relevant skills.
- I'd prefer a standard application, and I'll judge these more harshly than normal applications (you likely had much more time), but if you otherwise won't have time to apply I'd prefer to get these!
 - If you won't even have time to write an executive summary, I'd still rather get your application than nothing, but have an extremely high bar for those.
- If the previous work was done on your own and in ≤ 20 hours, but not *for* the application, this is obviously fine, and you can just treat it as a normal application project.

Defining the 20+2 hour time limit

- Not counted:
 - General prep (paper reading, tutorials), that you would have done before deciding on a project
 - My fairness principle here is "anything you could reasonably have done to learn mech interp on your own, before thinking of a problem to work on, is totally fine, because more experienced applicants could have done that already"
 - Generic tech set up, like renting and setting up a cloud GPU, that you'd need to do for most projects
 - Breaks
 - Time spent waiting for things to train (assuming you're doing something else during this time, eg training an SAE overnight)
 - Writing your answers to the MATS application form
- I consider any time you spend actively working towards the project goals to be within the 20 hour time limit. This includes (but is not limited to):

² If you're not a first author but made meaningful research contributions, please outline what they were in your application

- Writing code for your project
- Reading papers (chosen because they're relevant to your project)
- Analysing data/experimental results
- Thinking and planning time
- Writing up the google doc
- So the executive summary doesn't get super rushed, you can take another 2 hours for it.
 - I ask that you don't edit the rest of the write-up, and don't write any new experiment code, though you're welcome to write code to make new graphs/visualisations from data you already have, if it'll help present the results better
- You're encouraged to track your time with a tool like [Toggl](#) and include a screenshot with the application doc
- If you decide your project is doomed, you're welcome to give up and start a new one, and reset the timer

Advice for Good Applications

How to produce a good application in 20 hours

I recommend thinking of the application as a mini-research project. My standards are obviously lower than for a full paper, but the best applications look like small, self-contained research investigations. They essentially speedrun the process of identifying an interesting hypothesis, carefully testing it, and then clearly communicating the results. My blog posts on my research process ([Explore](#), [Understand](#), [Distill](#) and [Key Mindsets](#)) have more detail, but I've summarized the key ideas here.

Research Advice

There are three key phases to a good research project:

1. **Exploration:** The goal here is simply to **gain information and build intuition**. A common mistake is thinking this stage ends once you've picked a problem, e.g. from the list [below](#). In reality, much of a project is spent just figuring out what's going on.
 - a. You don't need a clear hypothesis yet. Often the best uses of time are things that expose you to lots of information.
 - b. **Get your hands dirty.** Try things like reading your data, giving your model interesting prompts, or seeing what a sparse autoencoder tells you.
 - c. This doesn't mean you don't have a plan. It means your plan is to maximize information gain per unit time. Constantly ask yourself: "**Have I learned anything in the last 30 minutes?** Is this direction still fruitful?"
2. **Understanding:** Once you have a hunch about what might be true, your goal is to **convince yourself it's true** with careful experiments.

- a. Keep a running doc with a list of your hypotheses. Alternate between designing an experiment to test one, running it, and analyzing the results.
 - Put key graphs and findings in your doc, as you learn more about hypotheses - you don't want to forget where a key experiment is!
 - b. It's crucial to keep track of the kind of claim you are trying to make.
 - Sometimes you want to give an existence proof (e.g., find an example of an interesting phenomenon), where cherry-picking is fine.
 - Other times, you want to argue a method is the right thing to do for a task, which requires comparing to baselines.
 - c. **Common mistakes:** Getting too excited and missing simple alternative explanations for your results; running a bunch of experiments that are only vaguely relevant instead of striving for **conclusive evidence**.
3. **Distillation:** This is where you turn your findings into something legible that can convince others. This means writing up your work clearly and honestly.
- a. **This is not an afterthought!** People often neglect the write-up, but it's crucial. **If I don't understand what you did, I will reject your application.**
 - b. Given the time limit, you won't achieve the full rigor of a published paper (e.g., large sample sizes, extensive baselines). That's fine! But the principles of providing clear evidence for your claims still apply.
 - Crucially, **avoid relying only on a few cherry-picked qualitative examples**—this is a major red flag.
 - And remember to compare to baselines, if applicable

Writing Advice

It's extremely important to have a good write-up! The advice in my post on [writing ML papers](#) may be helpful - obviously, I don't expect a formal paper, but the principles of clear communication apply.

- **Focus on a Narrative.** Don't just dump all your experiments. Structure your write-up around the one or two most interesting, concrete insights you found. What is the key story?
- **Quality over Quantity.** One interesting finding, well-explained and well-supported, is far better than ten superficial experiments.
- **Show Your Work.** Explain *why* you ran an experiment, not just *what* you did. What hypothesis were you testing? What were the possible outcomes? This reveals your thought process.
- **Your Reader Has Zero Context.** The "illusion of transparency" is a huge trap. Things that feel obvious to you will be completely new to your reader. Explain everything from the ground up. Define your terms. Label your graphs clearly.
- **Make Your Executive Summary Count.** It needs to stand on its own and convey the most important takeaways and a sketch of your key evidence. Don't make me hunt for the point or crucial details. Good graphs are a huge plus here.

Sanity-check your agent

This is the most important piece of advice in this doc. Modern agents will happily generate a plausible-looking research project - plausible hypotheses, plausible code, plausible graphs, plausible conclusions - that is subtly (or unsubtly) wrong. A crucial thing I am evaluating is whether you add value beyond me just prompting Fable myself. An application that is clearly "an agent did a project and a human forwarded it to me" will be rejected; I can get that myself, in twenty minutes, for free.

Sanity-checking is worth a lot of your time and care - I'd guess a meaningful fraction of your 20 hours. Concretely:

- Read the raw data. Read actual transcripts/rollouts, look at the actual prompts sent, look at datapoints the metric says are positive and check they really are. (This has always been good research advice - it's just far easier to skip now that an agent does the legwork.)
- Verify the load-bearing claims. For each key result: read the code that produced it, check the numbers in the write-up against the actual outputs, re-derive at least some of them independently (e.g. recompute a headline number with a fresh one-liner, or spot check by hand).
- Be suspicious of success. If the agent says an experiment worked, treat that as a hypothesis, not a result. Ask: what's the dumbest way this could be wrong? (Data leakage, trivial baseline matching it, the metric not measuring what you think, the model in the loop gaming your grader...) Then check.
- Design experiments yourself. Agents are great at executing experiments and terrible at noticing that the experiment doesn't test the hypothesis. The experimental design, the controls and baselines, and the interpretation of results should be yours.
- Document your checking in the write-up. Tell me what you verified and how - "I read 30 transcripts and confirmed the probe's positives were real" is strong evidence of research skill. In past rounds, some otherwise-promising applications were sunk because the write-up claimed things the applicant's own numbers contradicted - I do check.

For what it's worth, the data agrees: in the last round, applicants who described using LLMs agentically (Claude Code etc.) were accepted at ~3x the rate of those who mainly used LLMs for writing polish. The tools are a genuine edge - for the people who stay in control of them.